

Fraud Detection

Machine Learning Lifecycle

A step-by-step framework for identifying and stopping fraudulent transactions using machine learning — from business objective to live monitoring.

7

Phases

14

Activities

Real-time

Inference

MLOps

Practices

Table of Contents

Phase 1 — Problem Definition	3
Phase 2 — Data Collection	4
Phase 3 — Data Preparation & Cleaning	5
Phase 4 — Model Training	6
Phase 5 — Model Evaluation	7
Phase 6 — Deployment	8
Phase 7 — Monitoring & Retraining	9

1

Phase 1 | Problem Definition

THE CONCEPT

Outlining the core business goal and locking down the fraud-detection objective before any data work begins.

THE ACTION

The system identifies the target metric: flag suspicious transactions in real time so the bank can block fraudulent payments before settlement. Success is defined as maximising recall on fraud cases while keeping false-positive rates below 0.5% to avoid disrupting genuine customers.

REFERENCES

[1] [linkedin.com](#) — ML Life Cycle overview

[2] [carnritegroup.com](#) — Scaling ML successfully

[3] [certlibrary.com](#) — Data science project checklist

2

Phase 2 | Data Collection

THE CONCEPT

Gathering all raw historical transaction logs, account records, and device fingerprints into a centralised data lake.

THE ACTION

The system pulls card-swipe logs, online purchase records, IP geolocation data, device IDs, and account-age profiles from the payment gateway database. Both labelled fraud cases (charge-backs) and legitimate transactions are collected to build a representative training corpus.

3

Phase 3 | Data Preparation & Cleaning

THE CONCEPT

Scrubbing duplicates, fixing missing fields, encoding categories, and addressing extreme class imbalance before feeding data to the model.

THE ACTION

The system drops duplicate transaction IDs, imputes missing merchant-category codes, converts timestamps into hour-of-day and day-of-week features, and applies SMOTE oversampling so the rare fraud class is not drowned out by legitimate transactions during training.

REFERENCES

4

Phase 4 | Model Training

THE CONCEPT

Feeding the balanced, cleaned feature matrix into an algorithm so it learns the statistical signature of a fraudulent transaction.

THE ACTION

The computer runs the `.fit()` loop over millions of labelled transactions, training an XGBoost classifier to map spending velocity, location anomaly scores, and device-mismatch flags to a fraud-probability output between 0 and 1.

REFERENCES

[1] dagster.io — ML pipeline orchestration

[2] baeldung.com — Classification model training

[3] buildkite.com — Reproducible ML pipelines

5

Phase 5 | Model Evaluation

THE CONCEPT

Testing the trained model against a held-out exam dataset to measure its precision, recall, and AUC before any live deployment.

THE ACTION

The system evaluates predictions on an unseen test set, generating a confusion matrix and ROC curve. The model must achieve a recall above 90% on confirmed fraud cases and an AUC above 0.97 to clear the deployment gate.

REFERENCES

[1] refontelearning.com — Model evaluation techniques

6

Phase 6 | Deployment

THE CONCEPT

Launching the approved model onto cloud inference servers so it can score every incoming transaction in milliseconds.

THE ACTION

The model is containerised with Docker and served via a FastAPI endpoint behind the payment gateway. Every card-swipe triggers a scoring call; transactions above the risk threshold are held for secondary review or auto-declined in under 200 ms.

7

Phase 7 | Monitoring & Retraining

THE CONCEPT

Tracking live prediction logs and updating the model whenever fraudsters adapt their tactics and concept drift degrades performance.

THE ACTION

The system monitors daily precision-recall dashboards in Grafana. When recall drops below 88% for three consecutive days, an automated retraining job pulls the latest charge-back data and re-fits the model, keeping fraud detection sharp against evolving attack patterns.

REFERENCES

[1] [elearningindustry.com](#) — Streamlining operational monitoring

[2] [springer.com](#) — MLOps and model lifecycle management